Structural Stagnation Detection with Deployment Analysis

Anonymous

May 15, 2026

Abstract

We present a framework for real-time LLM agent stagnation monitoring that connects detection to deployment-oriented intervention analysis. Our central finding is that lightweight structural features (<5ms per window) dominate zero-shot cross-agent transfer: 30 features achieve 0.995 in-domain and 0.967 cross-agent F1 with no target domain data, outperforming a zero-shot neural probe by +37.2pp on a matched-scope evaluation (95% CI [34.1, 40.3], $p<0.001$) and the best LLM judge by +23.9pp on cross-agent data. A deployable 2-LF configuration (without privileged task outcomes) maintains 0.96 F1 on high-confidence labels, though coverage is agent-architecture dependent (2.7% on OpenHands vs. 66.5% on SWE-Agent). We translate detection into preliminary deployment analysis: retrospective compute waste quantification reveals that 27.3% of agent compute is consumed after stagnation onset, and early termination simulation shows structural detectors can save 17.8% compute while preserving 89.4% task success at $k=3$ consecutive stagnation windows. A Pareto cost-effectiveness analysis (AUC 0.70 vs. random 0.50) motivates future deployment work. Three temporal label model variants all fail to improve over simple majority vote, providing a well-characterized negative result. Human-annotated gold validation (328 SWE-Agent windows, 3 annotators) confirms structural advantage (GB 0.821 vs. GRU Probe 0.788, $p=0.042$).

1 Introduction

LLM-based coding agents increasingly tackle complex, multi-step software engineering tasks [1, 2]. A prevalent failure mode is *semantic stagnation*: the agent executes actions that are individually non-repetitive and locally plausible but collectively fail to make progress toward the task goal. Unlike syntactic loops detectable by cycle checkers, semantic stagnation manifests as diverse but unproductive action sequences that can persist for many steps, consuming substantial compute.

Existing approaches address fragments of this problem. Cycle detectors identify structural repetitions but miss semantically diverse stagnation. Process reward models (PRMs) provide step-level training signals but require outcome labels and RL pipelines [3]. LLM-as-judge approaches incur high inference costs and, as we show, degrade substantially in cross-agent transfer. Self-reflection mechanisms depend on the judgment of the very model that is stuck.

We propose a framework that addresses stagnation monitoring from detection through preliminary deployment analysis. Our contributions are:

1. **Structural features dominate zero-shot cross-agent transfer.** On a matched-scope evaluation controlling for prevalence and evaluation conditions, structural features outperform a zero-shot neural probe by +37.2pp ($p<0.001$) and the best LLM judge by +23.9pp. This establishes structural features as the default choice for zero-data deployment scenarios (§4.2).

2. **LF temporal dynamics and label quality as transfer bottleneck.** Labeling functions exhibit agent-specific temporal roles, and we demonstrate that label quality—not model architecture—is the dominant factor in cross-agent transfer. A prevalence-matched ablation reveals that the cmd_rep LF's primary contribution is coverage (-29.4pp when removed, $p\approx 0$), not label precision (§4.3).

3. **Temporal label model: a well-motivated negative finding.** Three temporal label model implementations all fail to improve over majority vote, despite well-motivated design. We provide detailed failure analysis constraining future research (§4.4).

4. **Deployment analysis: compute waste quantification and preliminary early termination.** Retrospective analysis reveals 27.3% compute waste post-stagnation onset. Early termination simulation and Pareto cost-effectiveness analysis motivate future online deployment (§5).

2 Related Work

Agent Failure Detection. Cycle detectors such as CDCS+CDSA and SpecRA capture structural repetitions but miss semantically diverse stagnation. TrajAD applies anomaly detection to trajectories but operates at the action level without window-level temporal modeling. SlopCodeBench [4] quantifies coding agent degradation over long tasks, finding structural erosion in 77% and verbosity increase in 75.5% of trajectories, motivating our stagnation detection framework.

Process Reward Models. AgentPRM [3] trains step-level PRMs via TD/GAE to evaluate "promise and progress," but requires outcome labels and RL infrastructure. Our approach uses weak supervision with no outcome labels for monitoring (not training), operating at the window level to capture temporal dynamics.

LLM Self-Reflection and Monitoring. Reflexion, LATS, and Tree of Thoughts rely on the stuck model's own judgment. Recent work on sequence-level agent monitoring argues single-step analysis is insufficient. We provide external, lightweight monitoring independent of the agent's self-assessment.

Weak Supervision. Data programming [5] combines noisy labeling functions via generative models. We extend this paradigm to sequential agent trajectories, discovering that standard i.i.d. assumptions are violated by LF temporal dynamics—yet temporal label models fail to exploit this structure.

3 Method

3.1 Problem Formulation

Given an agent trajectory $\tau = (a_1, a_2, \ldots, a_T)$ of actions, we segment it into overlapping windows of size k (default k=5). For each window $w_t = (a_t, \ldots, a_{t+k-1})$, the task is binary classification: *stagnation* (the window lacks goal-directed progress) vs. *progress*.

3.2 Labeling Functions

We design three labeling functions (LFs) operating on different signal modalities:

- **cmd_rep** (λ_1): Detects command repetition patterns within the window. High precision (97.91%), low false positive rate (0.15%).
- **file_diff** (λ_2): Measures file-level change rate across the window. Captures stagnation as absence of meaningful code modifications.
- **outcome_backpropagation** (λ_3, privileged): Backpropagates task outcome labels to late-trajectory windows. Provides distant supervision but requires outcome information unavailable at deployment time.

Labels are aggregated via majority vote (MV) with a confidence threshold: windows with $\geq$2 LF agreement (conf$\geq$2) form the high-confidence training set.

Table 1: Matched-scope comparison on OpenHands (N=6,122, stag prevalence 19.3%). Structural GB trained on SWE-Agent, evaluated zero-shot.

Model	Macro F1	$F1_{stag}$	$F1_{prog}$	Acc.
Structural GB	0.970	0.951	0.989	0.978
GRU Probe (0-shot)	0.598	0.308	0.888	0.672
Δ (pp)	+37.2	+64.3	+10.1	+30.6

3.3 Structural Features

We extract 30 structural features per window, organized into categories: repetition rates (command, argument, output), file change statistics (files modified, diff sizes, churn), output length patterns (mean, variance, trend), and temporal indicators (position in trajectory, elapsed steps). These features capture domain-invariant patterns of stagnation without relying on action semantics.

3.4 Models

Structural classifiers. Gradient Boosted Trees (GB) and Random Forest (RF) trained on the 30 structural features.

Neural probe. A GRU-based probe operating on trajectory embeddings from Qwen3-Embedding-0.6B. Encoder-agnostic: ablation across BGE, E5, and GTE encoders all achieve $>$0.90 F1.

LLM-as-judge baselines. Claude (zero-shot and few-shot-CoT) and GPT-5.5 (zero-shot) prompted to classify each window.

3.5 2-LF Deployable Configuration

For real-time deployment, we remove the privileged λ_3 (outcome_backpropagation), retaining only cmd_rep + file_diff. The 2-LF configuration requires no task outcome information and can operate during agent execution.

4 Experiments: Stagnation Detection

4.1 Data

We evaluate on trajectories from two coding agents on the SWE-bench benchmark: **SWE-Agent** (1,683 instances, 54,809 windows) and **OpenHands** (3,409 instances, 150,652 windows). Models are trained on SWE-Agent and evaluated zero-shot on OpenHands for cross-agent transfer.

4.2 Claim 1: Structural Features Dominate Zero-Shot Transfer

Matched-scope evaluation. To ensure fair comparison, we construct a matched subset of 6,122 OpenHands windows with stagnation prevalence (19.3%) matched to the overall distribution. Table 1 reports results on this controlled subset.

The structural GB achieves 0.970 macro F1 vs. 0.598 for the zero-shot probe, a gap of +37.2pp (bootstrap 95% CI [34.1, 40.3], $p<0.001$; McNemar $\chi^2=1823$, $p<0.001$). The gap is largest on the minority stagnation class (+64.3pp), where the probe's zero-shot representations fail to transfer. The matched-scope result is consistent with the full-set evaluation (Δ=36.9pp, difference 0.3pp), confirming that the advantage is not an artifact of evaluation subset selection.

Table 2: LLM judge cross-agent comparison. SWE F1 is in-domain; OH F1 is cross-agent zero-shot. Cost is per 5,000 windows.

Model	Config	SWE	OH	Degr.	Cost
Claude	0-shot	0.663	0.581	−8.2	$38.5
Claude	fs-CoT	0.799	0.728	−7.1	$112.4
GPT-5.5	0-shot	0.747	0.693	−5.4	$32.8
Struct. GB	—	0.995	0.967	−2.8	$0

Table 3: SWE-Agent gold set evaluation (N=328, stag prevalence 32.9%). TWV = temporal weighted vote probe.

Model	Macro F1	$\mathbf{F1}_{\mathbf{stag}}$	$\mathbf{F1}_{\mathbf{prog}}$	Δ MV	p
TWV Probe	0.835	0.798	0.872	+2.9	0.06
Structural GB	0.821	0.778	0.864	+1.5	0.18
MV Labels	0.806	0.762	0.850	—	—
Structural RF	0.804	0.758	0.850	−0.2	0.84
GRU Probe	0.788	0.738	0.838	−1.8	0.21

LLM judge comparison. Table 2 extends the comparison to LLM-as-judge baselines across both agents.

The best LLM judge (Claude few-shot-CoT, 0.728) trails the structural detector by +23.9pp on OpenHands. LLM judges degrade 5–8pp in cross-agent transfer vs. only 2.8pp for structural features. GPT-5.5 additionally suffers a 21.8% parse failure rate. The structural approach incurs zero marginal inference cost (<5ms per window).

Gold set validation. Table 3 validates on 328 human-annotated SWE-Agent windows (3 annotators).

On gold annotations, structural GB (0.821) significantly outperforms the GRU probe (0.788, +3.3pp, p=0.042), confirming that the structural advantage holds against neural baselines on human-validated labels. The GB vs. MV gap (+1.5pp) is not significant (p=0.18), likely due to limited sample size (328 vs. 491 in the full gold set).

4.3 Claim 2: LF Temporal Dynamics and Label Quality

Temporal dynamics. LFs exhibit systematic but agent-specific temporal roles. On SWE-Agent, cmd_rep leads stagnation onset by −1.87 steps, file_diff is near-concurrent (−1.2 steps), and outcome_backpropagation lags by +6.7 steps. On OpenHands, cmd_rep shifts from leading to lagging (+3.32 steps), while file_diff (−4.9 steps) and outcome_backpropagation (+18.1 steps) maintain cross-domain consistency. These dynamics motivate but ultimately fail to benefit temporal label models (§4.4).

Label quality as transfer bottleneck. Label audit revealed that correcting an OpenHands label version bias (stag_rate 6.56%→21.45%) caused probe 5% fine-tuning performance to jump from 0.570 to 0.850 (+28pp)—the corrected 5% surpasses the old 50% (0.688). The cmd_rep LF, previously mischaracterized as a noise source (71% FP under buggy labels), is actually a high-precision detector (precision 97.91%, FP rate 0.15%).

cmd_rep coverage contribution. A prevalence-matched ablation on the same test set (N=6,122, stag 19.3%) reveals cmd_rep's primary value: removing it does not change label quality on the intersection (2-LF and 3-LF labels are 100% consistent on 4,017 shared windows) but reduces available training data from 43K to 4K windows, resulting in a −29.4pp drop (McNemar χ^2=1551, $p\approx$0). The contribution is coverage, not precision.

Table 4: Compute waste audit by agent and resolution status. Tail% = fraction of instances with stagnation tail. Compute% = fraction of total compute in tails.

Agent	Subset	N	Tail%	Comp.%	Tok.%	Med.
SWE-Ag.	All	1683	66.8	30.2	28.7	8
SWE-Ag.	Unres.	$\sim$1599	70.2	38.5	36.1	9
SWE-Ag.	Res.	$\sim$84	8.3	2.1	1.9	2
OH	All	3409	60.1	24.1	22.8	10
OH	Unres.	$\sim$3239	63.2	31.8	30.1	11
OH	Res.	$\sim$170	5.9	1.8	1.6	2
Comb.	All	5092	62.3	27.3	25.8	9

4.4 Claim 3: Temporal Label Model Negative Finding

The systematic temporal dynamics of LFs (§4.3) motivate temporal label models that respect sequential structure. We implement three variants: (a) **TWV** (temporal weighted voting): harmful for most bandwidth parameters σ; the sole mildly positive σ=2 produces downstream degradation ($-$4.9pp in-domain, $-$21.7pp cross-agent); (b) **HMM-TLM**: overpredicts stagnation; (c) **Lag-aware HMM**: matches but does not exceed MV (0.8936). In the current 3-LF, low-conflict regime (conf$\geq$2 agreement 95.1%), temporal alignment gains are absorbed by noise introduction. Detailed failure analysis is in Appendix A.

5 Deployment Analysis

We translate stagnation detection into preliminary deployment analysis through compute waste quantification and early termination simulation. These results are offline retrospective analyses that motivate—but do not constitute—online deployment.

5.1 Compute Waste Quantification

We retrospectively analyze all trajectories to quantify *stagnation tails*: computation consumed from first stagnation onset to trajectory end.

Table 4 shows that approximately one-quarter of agent compute (27.3%) is consumed after stagnation onset. The waste is concentrated in unresolved trajectories (35.4% of their compute) while resolved trajectories have minimal tails (2.1%). SWE-Agent exhibits higher waste (30.2%) due to higher stagnation rates and shorter trajectories. Notably, 10.9–14.3% of trajectories show intermittent recovery (brief stagnation followed by resumed progress), indicating that stagnation is not always monotonic and early termination must account for false positive risk.

5.2 Early Termination Simulation

We simulate an early stopping policy that terminates agent execution after k consecutive windows predicted as stagnation by the structural GB detector. Table 5 reports the savings–preservation trade-off across threshold values.

At k=3, the policy saves 17.8% compute while preserving 89.4% task success (false termination rate 10.6%). The trade-off curve is steep: moving from k=2 to k=4, each unit increase reduces false terminations by $\sim$3pp but costs $\sim$4pp in savings. This reflects intermittent stagnation patterns where agents temporarily stall during successful problem-solving. Per-agent analysis reveals SWE-Agent benefits more (20.4% saved at k=3) than OpenHands (16.3%), consistent with SWE-Agent's higher stagnation rates. Compared to an oracle (22.5% saved, 100% preserved at k=3) and random baseline (65.2% preserved at 18% savings), the structural detector provides meaningful but bounded advantage.

Table 5: Early termination simulation using structural GB detector. Saved% = compute saved; Pres.% = task success preserved; FTR = false termination rate.

k	**Saved%**	**Pres.%**	N **Term.**	**Res. Kill**	**FTR**
1	28.3	77.6	3210	57	22.4%
2	22.1	84.3	2650	40	15.7%
3	17.8	89.4	2140	27	10.6%
4	14.6	91.8	1780	21	8.2%
5	12.2	93.3	1490	17	6.7%
7	7.8	96.1	1050	10	3.9%
10	4.1	98.4	620	4	1.6%

Table 6: Pareto AUC comparison (normalized). Higher AUC = better savings–preservation trade-off across all operating points.

Method	**AUC**	**vs. Random**	**vs. Oracle**
Oracle (outcome)	0.97	+0.47	—
Oracle (detector)	0.86	+0.36	−0.11
Structural GB	0.70	+0.20	−0.27
LLM Judge (best)	0.56	+0.06	−0.41
Probe (zero-shot)	0.52	+0.02	−0.45
Random	0.50	—	−0.47

These results represent preliminary analysis showing moderate savings potential. The steep precision–recall curve indicates that early termination requires careful threshold calibration and is not a drop-in solution.

5.3 Cost-Effectiveness Pareto Analysis

Table 6 compares normalized Pareto AUC across methods, quantifying cost-effectiveness over the full savings–preservation frontier.

The structural GB achieves AUC 0.70, providing meaningful advantage over uninformed baselines (+0.20 vs. random) but with substantial room remaining to the theoretical optimum (−0.27 vs. oracle outcome). The LLM judge (0.56) and zero-shot probe (0.52) perform near random in the intervention setting, reinforcing that structural features' detection advantage translates to deployment scenarios. The gap between structural GB and the oracle detector (0.16) suggests that improved intermittent stagnation handling (e.g., adaptive k or cooldown periods) could substantially improve the Pareto frontier. We present this analysis as motivating future deployment work rather than claiming deployment-ready intervention.

5.4 2-LF Deployable Configuration

For real-time deployment, the privileged λ_3 must be removed. Table 7 evaluates the 2-LF (cmd_rep + file_diff) configuration across settings.

The 2-LF configuration is deployable with caveats. On high-confidence windows where both LFs fire, performance is strong (0.960–0.968 F1) and labels are 100% consistent with the 3-LF configuration (4,017 shared windows). However, coverage is heavily agent-architecture dependent: cmd_rep fires frequently on SWE-Agent (66.5% coverage) but rarely on OpenHands (2.7%), because OpenHands uses different tool-calling conventions where command repetition manifests differently. Cross-agent degradation is modest (−2.7pp on OH all), but the coverage limitation means the 2-LF detector can only provide high-confidence judgments for a small fraction of OpenHands windows. Future work should design agent-agnostic LFs to address this coverage gap. The early termination simulation savings (§5.2)

Table 7: 2-LF vs. 3-LF cross-agent performance. Coverage = fraction of windows with conf$\geq$2 labels.

Config	Eval Set	N	GB F1	RF F1	Cov.
3-LF	OH conf$\geq$2	43376	0.992	0.996	28.8%
2-LF	OH 2-LF conf$\geq$2	4017	0.960	0.968	2.7%
2-LF	OH 3-LF conf$\geq$2	43376	0.945	0.938	28.8%
2-LF	OH all	150652	0.940	0.925	100%
2-LF	SWE in-domain	10243	0.978	0.978	66.5%

are based on the 3-LF detector; real deployment with 2-LF would yield lower savings on OpenHands-type agents.

6 Conclusion

We present a framework for LLM agent stagnation detection that demonstrates a clear hierarchy of monitoring approaches. The primary finding—that 30 lightweight structural features dominate zero-shot cross-agent transfer by +37.2pp over neural probes and +23.9pp over LLM judges—establishes structural features as the default choice when target-agent data is unavailable. Our analysis reveals that label quality, not model architecture, is the dominant cross-agent transfer bottleneck, and that labeling function contributions are primarily through coverage rather than precision.

The deployment analysis provides preliminary but promising evidence: 27.3% of agent compute is consumed post-stagnation, and a k=3 early termination policy can save 17.8% compute while preserving 89.4% success. The Pareto analysis (AUC 0.70 vs. random 0.50) motivates future online deployment research.

Limitations. (1) Cross-agent evaluation covers two SE coding agents; generalization to non-SE agents remains untested. (2) The 2-LF deployable configuration has agent-architecture-dependent coverage (2.7% on OpenHands). (3) The intervention simulation is entirely offline retrospective; online deployment with real agent framework integration is left to future work. (4) Structural features' near-perfect in-domain F1 (0.995) partially reflects circular evaluation: features derived from LF inputs can reconstruct LF labels. Cross-agent and gold-set evaluations mitigate but do not eliminate this concern. (5) Gold set validation is limited to SWE-Agent (N=328); cross-agent gold validation on OpenHands is needed.

A TLM Failure Analysis

TWV. Temporal weighted voting assigns time-decayed weights based on each LF's temporal profile. Across bandwidth parameters $\sigma \in \{0.5, 1, 2, 5, 10\}$, TWV is harmful for most σ. The sole mildly positive setting (σ=2) produces downstream classifier degradation: -4.9pp in-domain and -21.7pp cross-agent on structural GB trained with TWV labels vs. MV labels. TWV primarily modifies low-confidence (conf=1) labels, but the modifications move labels in the wrong direction—away from human annotation agreement.

HMM-TLM. A hidden Markov model treating the true stagnation state as latent and LF outputs as emissions. The HMM systematically overpredicts stagnation due to the asymmetric transition structure: stagnation states are "stickier" than progress states in the learned transition matrix, causing cascading false positives.

Lag-aware HMM. Extends HMM-TLM with per-LF emission lag parameters estimated from the temporal dynamics analysis. Achieves 0.8936 F1, matching but not exceeding MV. The lag parameters

correctly capture temporal offsets but provide insufficient additional signal in the low-conflict regime where 95.1% of conf$\geq$2 labels already agree.

Diagnosis. In the current 3-LF regime, temporal label models face a fundamental constraint: the LFs have low mutual conflict (95.1% agreement at conf$\geq$2), leaving little room for temporal alignment to improve. The models primarily affect low-confidence labels where temporal signal is weakest and noise is highest. This provides a key constraint for future work: temporal label models may require more LFs with higher conflict rates to demonstrate benefit.

References

[1] J. Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv:2405.15793*, 2024.

[2] X. Wang et al. OpenDevin: An open platform for AI software developers as generalist agents. *arXiv:2407.16741*, 2024.

[3] Z. Xi et al. AgentPRM: Process reward model for agent training. *ACM Web Conference*, 2026. arXiv:2511.08325.

[4] SlopCodeBench. Quantifying coding agent degradation in long-horizon tasks. *arXiv:2603.24755*, 2026.

[5] A. Ratner et al. Snorkel: Rapid training data creation with data programming. *VLDB*, 2017.